# SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Mock Interview)

| | |
|---|---|
| **Course Name** | Python Programming |
| **Course Code** | CSA0804 |
| **Name** | Monish.L |
| **Register Number** | 1972260035 |
| **Topic** | CO3-AT3-MOCK INTERVIEW |
| **Question** | Q4 — Python Set Operations (10 Marks) |

**Q4. [10 Marks] Demonstrate all six set operations: union (|), intersection (&), difference (-), symmetric difference (^), issubset (<=), and issuperset (>=) with A={1,2,3,4,5} and B={3,4,5,6,7}.**

---

## Answer

A Python set is an unordered collection of unique, hashable elements that supports fast membership testing and provides built-in operators that directly mirror the mathematical set operations. Given A = {1, 2, 3, 4, 5} and B = {3, 4, 5, 6, 7}, each of the six operations is demonstrated below.

### *1. Union (|)*

A | B returns a new set containing every element that appears in A, in B, or in both, with duplicates naturally collapsed since a set cannot hold repeated values. Result: A | B = {1, 2, 3, 4, 5, 6, 7}.

### *2. Intersection (&)*

A & B returns only the elements common to both sets. Result: A & B = {3, 4, 5}.

### *3. Difference (-)*

A - B returns the elements present in A but not in B; this operation is not symmetric, so B - A gives a different result. Result: A - B = {1, 2}, while B - A = {6, 7}.

### *4. Symmetric difference (^)*

A ^ B returns the elements that are in exactly one of the two sets, but not in both — equivalent to (A - B) union (B - A), or the union minus the intersection. Result: A ^ B = {1, 2, 6, 7}.

### *5. issubset (<=)*

A <= B (or A.issubset(B)) checks whether every element of A is also present in B. Since A contains 1 and 2, which are not in B, A is not a subset of B. Result: A <= B is False. As a further example, {1, 2, 3} <= A is True, since every element of {1, 2, 3} is present in A.

### *6. issuperset (>=)*

A >= B (or A.issuperset(B)) checks whether A contains every element of B. Since B contains 6 and 7, which are not in A, A is not a superset of B. Result: A >= B is False. As a further example, A >= {1, 2, 3} is True, since A contains all of 1, 2, and 3.

## Python Program

```python
"""
CO3-AT3 - Mock Interview
Q4: Python Set Operations [10 Marks]
------------------------------------------------------------
Demonstrates union, intersection, difference, symmetric
difference, issubset, and issuperset using A and B.
"""

A = {1, 2, 3, 4, 5}
B = {3, 4, 5, 6, 7}

print("A:", A)
print("B:", B)

# 1. Union
print("Union (A | B):", A | B)

# 2. Intersection
print("Intersection (A & B):", A & B)

# 3. Difference
print("Difference (A - B):", A - B)
print("Difference (B - A):", B - A)

# 4. Symmetric difference
print("Symmetric difference (A ^ B):", A ^ B)

# 5. issubset
print("A issubset B (A <= B):", A <= B)
print("{1,2,3} issubset A:", {1, 2, 3} <= A)

# 6. issuperset
print("A issuperset B (A >= B):", A >= B)
print("A issuperset {1,2,3}:", A >= {1, 2, 3})
```

## Sample Output

```
A: {1, 2, 3, 4, 5}
B: {3, 4, 5, 6, 7}
Union (A | B): {1, 2, 3, 4, 5, 6, 7}
Intersection (A & B): {3, 4, 5}
Difference (A - B): {1, 2}
Difference (B - A): {6, 7}
Symmetric difference (A ^ B): {1, 2, 6, 7}
A issubset B (A <= B): False
{1,2,3} issubset A: True
A issuperset B (A >= B): False
A issuperset {1,2,3}: True
```

## Conclusion

Python's set type provides direct operator support for all standard mathematical set operations, making set-based logic concise and readable. With A = {1, 2, 3, 4, 5} and B = {3, 4, 5, 6, 7}, the union combines all distinct elements from both sets, the intersection isolates the shared elements {3, 4, 5}, the difference operators show what belongs to one set but not the other, and the symmetric difference captures everything that is not shared. The issubset and issuperset checks confirm that neither A nor B fully contains the other, since each has elements the other lacks, while smaller subsets such as {1, 2, 3} correctly test as being contained within A. Together these six operations demonstrate how Python sets closely follow standard set theory while remaining simple to use in code.